



MediSync AI: Monorepo Architecture Blueprint

This guide provides the configuration files and structural roadmap to set up a unified codebase for the **MediSync AI** project[cite: 1]. We will utilize **pnpm Workspaces** and **Turborepo** to manage our package dependencies, run tasks in parallel, and share utilities across our applications.

Workspace Directory Map

Ensure your project files are organized exactly as shown below:

```
medisync-ai/
├── apps/
│   ├── backend/           # Node.js + Express API Gateway
│   ├── mobile-app/        # React Native Patient Application
│   └── web-portals/        # React.js + Tailwind CSS Admin Portal
├── packages/
│   └── shared-types/       # Common TypeScript interfaces & schemas
├── package.json            # Root workspace configuration
├── pnpm-workspace.yaml     # Workspace packages declaration
└── turbo.json              # Turborepo execution pipeline configuration
```

1. Root Workspace Configuration

This file configures the root of your project, enabling workspace dependency management and mapping global scripts to build and run all sub-projects.

```
{
  "name": "medisync-ai",
  "private": true,
  "version": "1.0.0",
  "scripts": {
    "dev": "turbo run dev",
    "build": "turbo run build",
    "lint": "turbo run lint",
    "clean": "turbo run clean"
  },
  "devDependencies": {
    "turbo": "^2.0.0"
  },
  "engines": {
    "node": ">=18.0.0"
  },
  "packageManager": "pnpm@9.0.0"
}
```

2. Workspace Declaration (pnpm-workspace.yaml)

This configuration registers your `apps/` and shared library folders as active packages within the

workspace pipeline.

```
packages:
  - "apps/*"
  - "packages/*"
```

3. Build & Execution Pipeline (turbo.json)

Turborepo acts as the execution engine for the workspaces. It orchestrates cache logs and schedules parallel tasks (e.g., launching your API server and React bundler at the same time).

```
{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": [".next/**", "dist/**", "build/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "lint": {
      "dependsOn": ["^lint"]
    },
    "clean": {
      "cache": false
    }
  }
}
```

4. Shared Types Sub-Package (packages/shared-types)

To maintain structural symmetry between your API backend, patient mobile app, and doctor portal[cite: 1], create a lightweight, shareable TypeScript module for system interfaces and request/response payloads.

packages/shared-types/package.json

```
{
  "name": "@medisync/shared-types",
  "version": "1.0.0",
  "private": true,
  "main": "./dist/index.js",
  "types": "./dist/index.d.ts",
  "scripts": {
    "build": "tsc"
  },
}
```

```
"devDependencies": {  
  "typescript": "^5.0.0"  
}
```

packages/shared-types/src/index.ts

```
// Shared entities reflecting your PostgreSQL models and API contracts  
  
export interface UserProfile {  
  id: string;  
  fullName: string;  
  email: string;  
  role: 'PATIENT' | 'DOCTOR' | 'PHARMACY';  
}  
  
export interface Prescription {  
  id: string;  
  patientId: string;  
  doctorName: string;  
  digitizedNotes: string; // Structured output from Gemini Multimodal OCR  
  suggestedGenericAlternatives: string[];  
  createdAt: string;  
}  
  
export interface AlarmSchedule {  
  id: string;  
  patientId: string;  
  medicineName: string;  
  dosage: string;  
  triggerTimes: string[]; // Alarm hours  
  isActive: boolean;  
}  
  
export interface StockCheckMessage {  
  id: string;  
  senderId: string;  
  receiverId: string;  
  medicineQuery: string;  
  isAvailable: boolean;  
  timestamp: string;  
}
```

Integration Workflow: How to Use It

1. **Install Dependencies Globally:** Run `pnpm install` in your root directory. It will recursively link and optimize standard packages across all apps.
2. **Inject Shared Types:** To use the shared contracts in your `mobile-app` or `backend` folders, simply add `"@medisync/shared-types": "workspace:*"` to their local `package.json` dependencies.

3. **Run the Whole Project Locally:** Simply run `pnpm dev` in the root folder. Turborepo will start:
 - The backend server on its local port
 - The doctor web portal dashboard[cite: 1]
 - The React Native Metro Bundler for your mobile app[cite: 1]